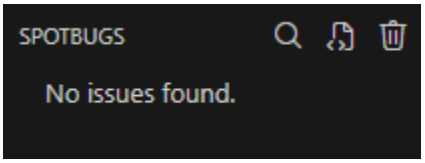
26/12/2025

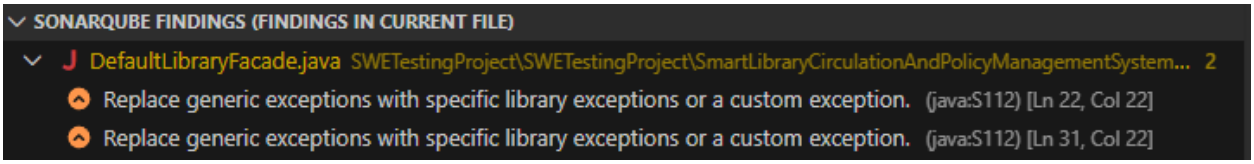
Static Testing Documentation

- Classes:

1. DefaultLibraryFacade:

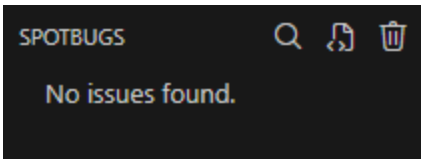
- **SonarQube** flags DefaultLibraryFacade because it uses overly generic exceptions such as Exception or Throwable in throws declarations and catch blocks. This makes it difficult for callers to understand what actually failed and can lead to important exceptions (for example, InterruptedException) being swallowed.
- To fix this, declare the specific exceptions that can occur (e.g., IOException, SQLException) or introduce a small domain-specific exception (such as LibraryFacadeException) and wrap the original exception as its cause. Avoid using catch (Exception); instead, catch only the exceptions you can meaningfully handle. If InterruptedException is caught, either rethrow it or restore the interrupt status using Thread.currentThread().interrupt().
- **SpotBugs** does not report an issue for this class, which is expected. SpotBugs focuses on detecting concrete bug patterns, whereas SonarQube also enforces design and maintainability rules such as S112.

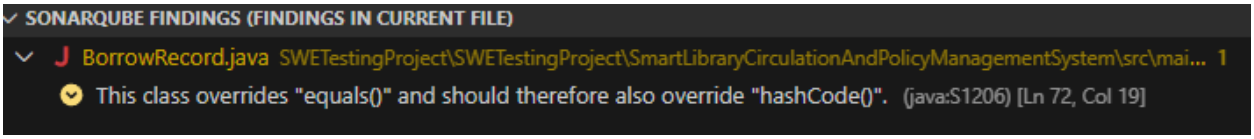




2. BorrowRecord:

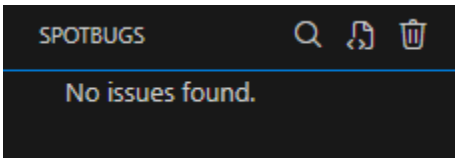
- **SonarQube** is issuing a warning because BorrowRecord overrides equals() but does not override hashCode(). In Java those two must be consistent: if equals() says two objects are equal, they must return the same hash code. If you leave hashCode() as the default (identity-based), equal objects can end up in different buckets in hash-based collections like HashSet or HashMap, which leads to lost entries, failed lookups, and subtle runtime bugs.
- To fix it, add a hashCode() that uses the same fields you use in equals(). Make sure you include exactly the same properties and be careful about using mutable fields if instances will be stored in hash-based collections. You can have your IDE generate it, use Objects.hash(field1, field2), or use a project utility like Lombok's @EqualsAndHashCode if that's allowed.
- **SpotBugs** did not flag any issues for this class.

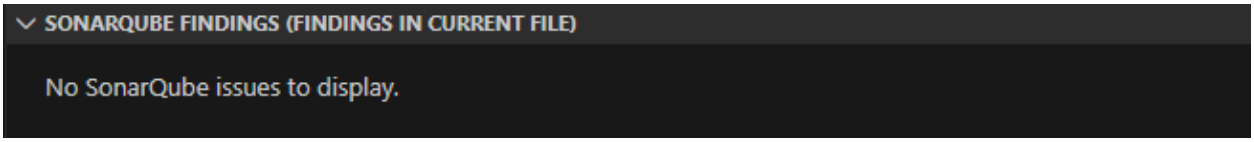




3. LibraryItem:

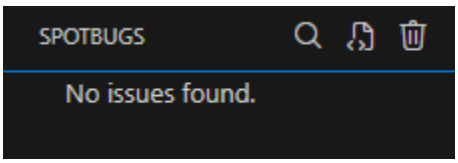
- **SonarQube** and **SpotBugs** didn't report any issues for LibraryItem class. SonarQube raised no rules and SpotBugs found no bug patterns.
- That doesn't mean the class is flawless, only that these tools didn't flag anything.

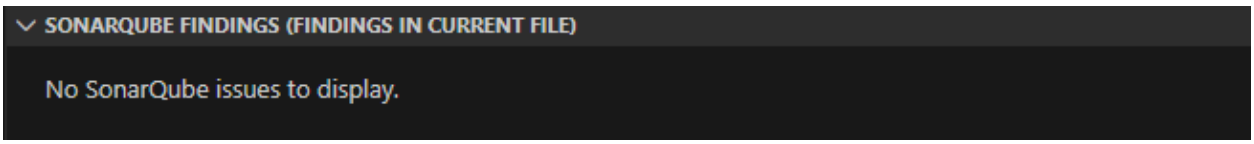




4. Member:

- **SonarQube** and **SpotBugs** didn't report any issues for Member class. SonarQube raised no rules and SpotBugs found no bug patterns.
- That doesn't mean the class is flawless, only that these tools didn't flag anything.

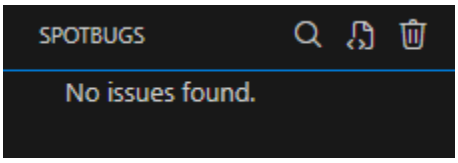


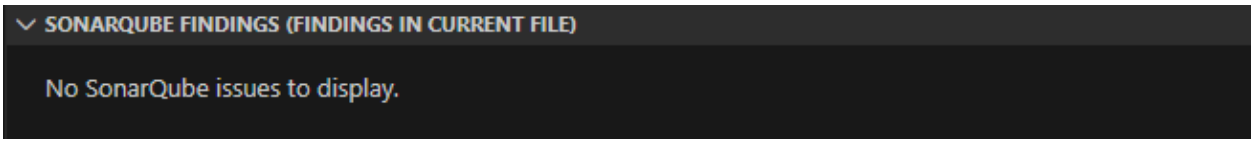


5. StubAlertService:

- **SonarQube** and **SpotBugs** didn't report any issues for StubAlertService class. SonarQube raised no rules and SpotBugs found no bug patterns.

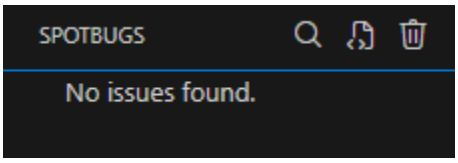
- That doesn’t mean the class is flawless, only that these tools didn’t flag anything.

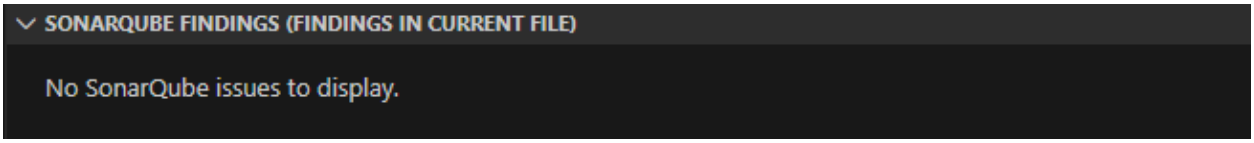




6. DefaultBorrowPolicy:

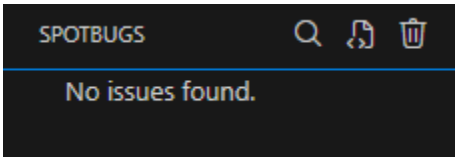
- **SonarQube** and **SpotBugs** didn’t report any issues for DefaultBorrowPolicy class. Sonar raised no rules and SpotBugs found no bug patterns.
- That doesn’t mean the class is flawless, only that these tools didn’t flag anything.

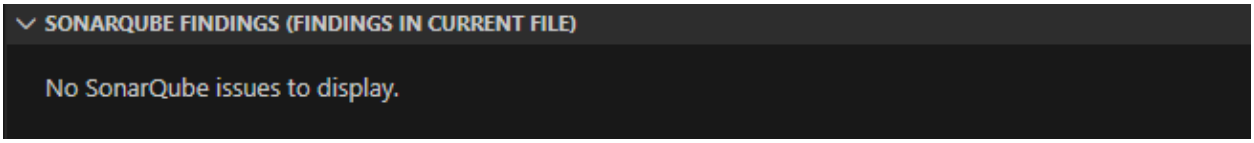




7. DefaultPenaltyPolicy:

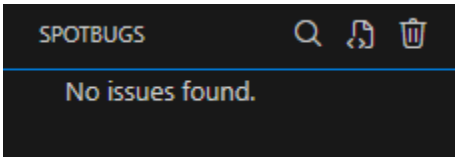
- **SonarQube** and **SpotBugs** didn’t report any issues for DefaultPenaltyPolicy class. SonarQube raised no rules and SpotBugs found no bug patterns.
- That doesn’t mean the class is flawless, only that these tools didn’t flag anything.

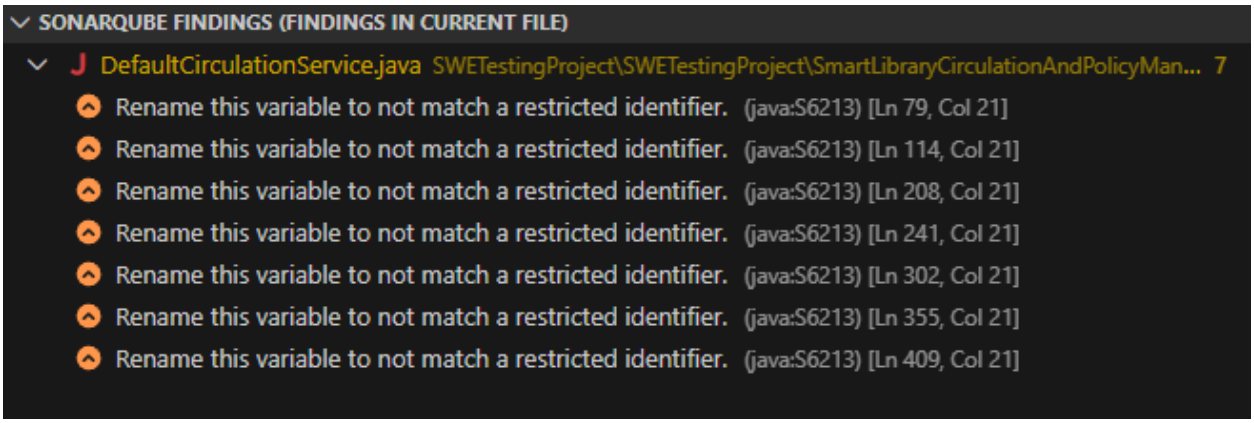




8. DefaultCirculationService:

- **SonarQube** is issuing a warning because several local variables in DefaultCirculationService match a restricted identifier naming pattern. These names can be confusing or misleading, for example being too short, ambiguous, or resembling class, framework, or reserved identifiers.
- This can be fixed by renaming the affected variables to clear and descriptive camelCase names that reflect their purpose. Single-letter or ambiguous names should be avoided, and variable names should not reuse class or known framework identifiers. Renaming should be done using the IDE's refactoring tools to safely propagate the changes. If a variable name is intentionally chosen and properly documented, the rule may be suppressed for that specific line or adjusted in the SonarQube configuration, although renaming is the preferred solution.
- **SpotBugs** did not report any issues for this class.

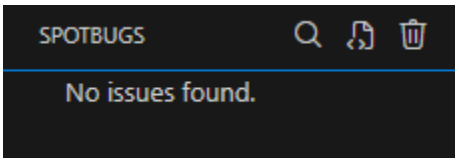


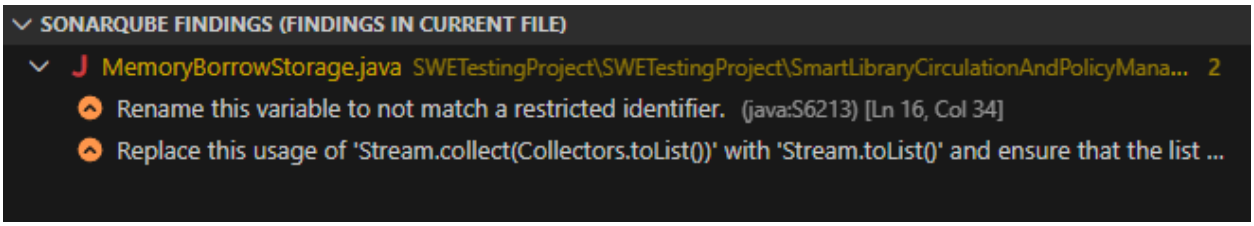


9. MemoryBorrowStorage:

- **SonarQube** is issuing a warning because MemoryBorrowStorage contains two issues. First, rule **S6213** reports that some local variable names match a restricted identifier pattern, making them unclear or potentially misleading (for example, being too short, clashing with framework or common identifiers, or shadowing other names). Second, Sonar suggests replacing Stream.collect(Collectors.toList()) with Stream.toList().
- The naming issue can be fixed by renaming the affected variables to clear and descriptive camelCase names that reflect what the variable represents (for example, borrowList, borrowRecord, currentBorrowCount, index, or requestId). Renaming should be done using the IDE's refactoring tools to ensure consistency across the codebase.
  The stream-related issue can be fixed by using Stream.toList() if the project is running on Java 16 or newer and an unmodifiable list is acceptable. If a mutable list is required, Collectors.toList() should be kept, or the result of stream.toList() should be wrapped in a new ArrayList<>. -

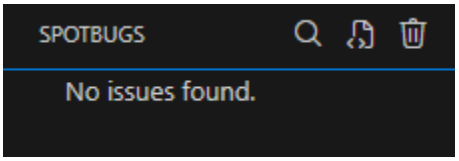
- **SpotBugs** did not report any issues for this class.

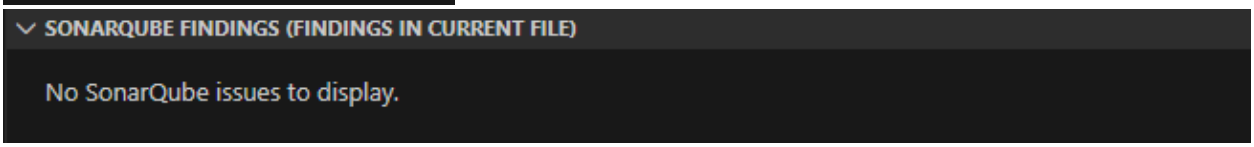




10. MemoryItemStorage:

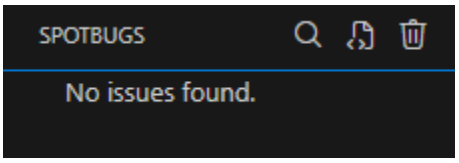
- **SonarQube** and **SpotBugs** didn't report any issues for MemoryItemStorage class. SonarQube raised no rules and SpotBugs found no bug patterns.
- That doesn't mean the class is flawless, only that these tools didn't flag anything.

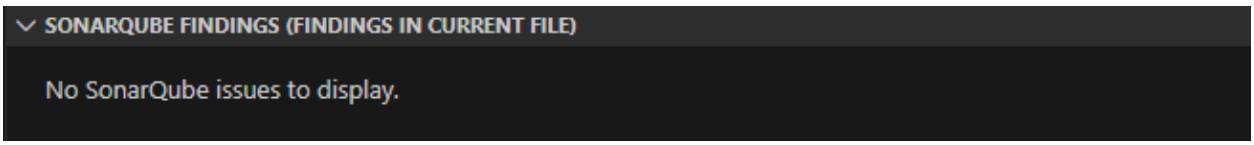




11. MemoryMemberStorage:

- **SonarQube** and **SpotBugs** didn't report any issues for MemoryMemberStorage class. SonarQube raised no rules and SpotBugs found no bug patterns.
- That doesn't mean the class is flawless, only that these tools didn't flag anything.

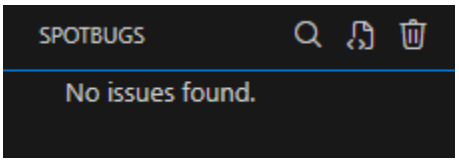


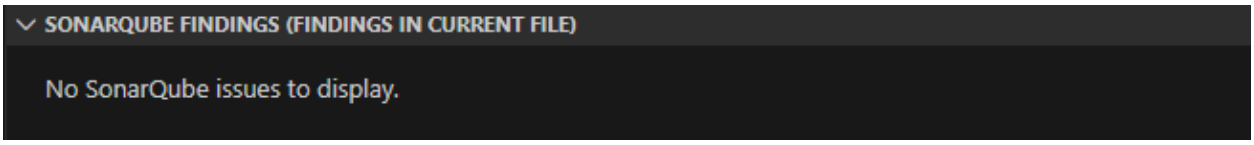


- Interfaces:

1. LibraryFacade:

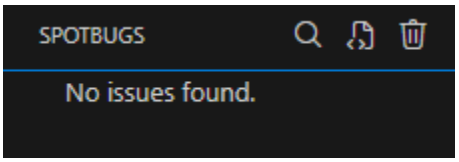
- **SonarQube** and **SpotBugs** didn't report any issues for LibraryFacade interface. SonarQube raised no rules and SpotBugs found no bug patterns.
- That doesn't mean the interface is flawless, only that these tools didn't flag anything.

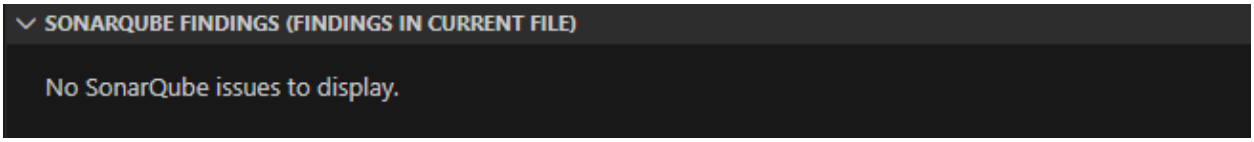




2. AlertService:

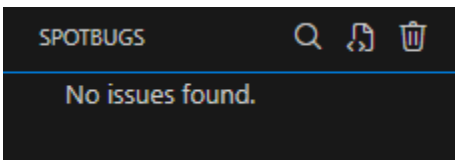
- **SonarQube** and **SpotBugs** didn't report any issues for AlertService interface. SonarQube raised no rules and SpotBugs found no bug patterns.
- That doesn't mean the interface is flawless, only that these tools didn't flag anything.

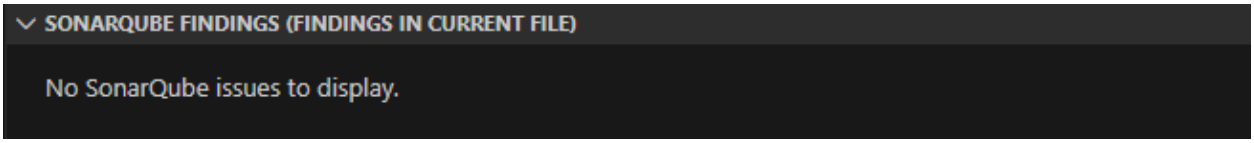




3. BorrowPolicy:

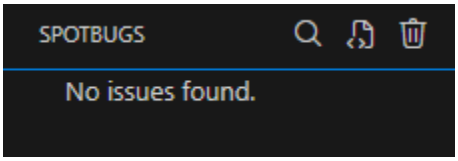
- **SonarQube** and **SpotBugs** didn't report any issues for BorrowPolicy interface. SonarQube raised no rules and SpotBugs found no bug patterns.
- That doesn't mean the interface is flawless, only that these tools didn't flag anything.

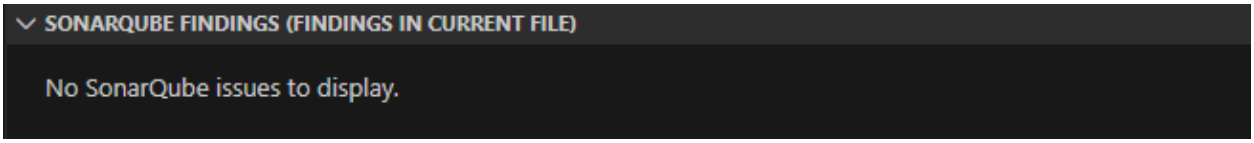




4. PenaltyPolicy:

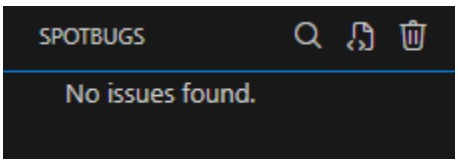
- **SonarQube** and **SpotBugs** didn't report any issues for PenaltyPolicy interface. SonarQube raised no rules and SpotBugs found no bug patterns.
- That doesn't mean the interface is flawless, only that these tools didn't flag anything.

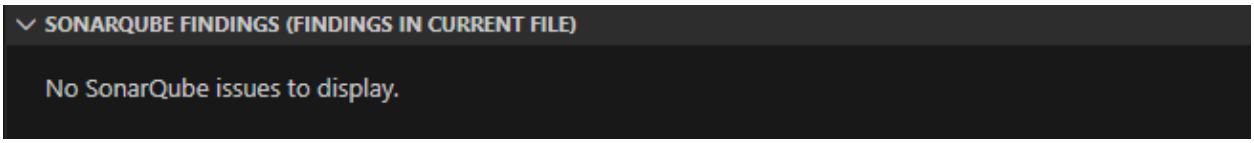




5. CirculationService:

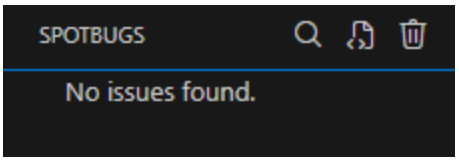
- **SonarQube** and **SpotBugs** didn't report any issues for CirculationService interface. Sonar raised no rules and SpotBugs found no bug patterns.
- That doesn't mean the interface is flawless, only that these tools didn't flag anything.

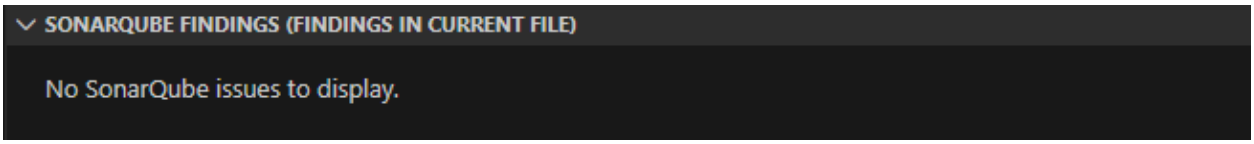




6. ItemStorage:

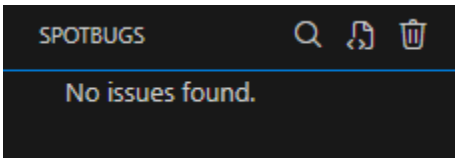
- **SonarQube** and **SpotBugs** didn't report any issues for ItemStorage interface. SonarQube raised no rules and SpotBugs found no bug patterns.
- That doesn't mean the interface is flawless, only that these tools didn't flag anything.

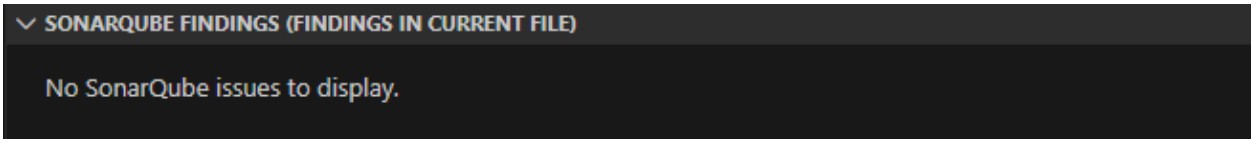




7. MemberStorage:

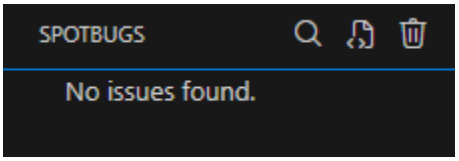
- **SonarQube** and **SpotBugs** didn't report any issues for MemberStorage interface. SonarQube raised no rules and SpotBugs found no bug patterns.
- That doesn't mean the interface is flawless, only that these tools didn't flag anything.

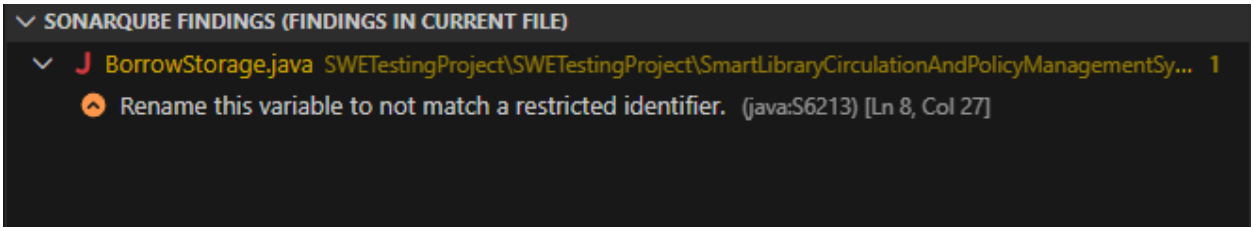




8. BorrowStorage:

- **SonarQube** is issuing a warning because rule S6213 is raised in BorrowStorage, indicating that a local variable name matches a restricted identifier pattern. This means the variable name may be confusing or risky, for example being too short, shadowing important identifiers, or resembling reserved or framework names.
- This can be fixed by renaming the affected variable to a clear and descriptive camelCase name that reflects its purpose. Single-letter or ambiguous names should be avoided, and the IDE's rename refactoring tools should be used to ensure the change is applied safely throughout the code. If the variable name is intentionally chosen and properly documented, the rule may be suppressed for that specific line, although renaming remains the preferred solution.
- **SpotBugs** did not report any issues for this file.





- Enumerations:

1. MembershipType:

- **SonarQube** and **SpotBugs** didn't report any issues for MembershipType enumeration. SonarQube raised no rules and SpotBugs found no bug patterns.
- That doesn't mean the enumeration is flawless, only that these tools didn't flag anything.

